Homework 09

Collecting Beepers

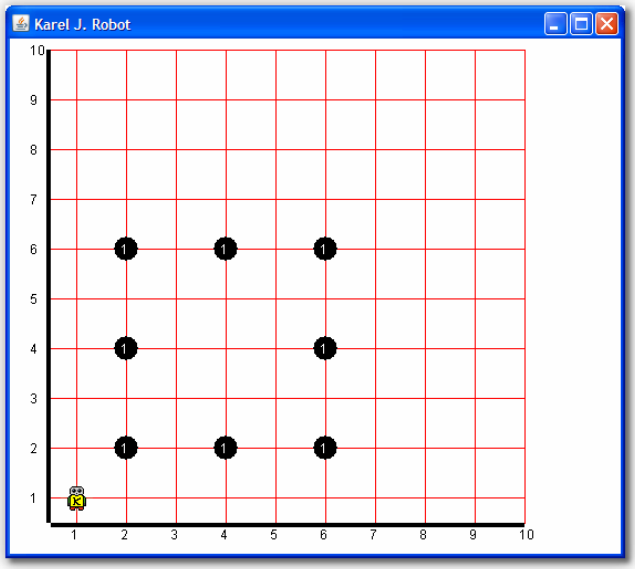
Goal

In this lab assignment, you will be writing a simple Java program to create a robot object called `karel`. Your robot will start off in a world containing a series of beepers arranged in a square. Your goal is to instruct your robot to pick up all the beepers and then turn himself off. You will also write simple test cases to demonstrate that your program works correctly.

Learning Objectives
• Familiarity with BlueJ projects • Familiarity with creating a simple class • Familiarity with writing simple method calls • Familiarity with Karel's world • Familiarity with interactive methods in BlueJ • Exposure to recording simple unit tests • Exposure to the Checkstyle style checker

Part I: Basic Robot Control

In Part I of this assignment, you will write a robot program that will direct Karel to navigate around his world and pick up all the beepers. The starting point of the world is shown below.



Procedure

1. **Create a Java class for your solution**

   a. Start BlueJ on your machine. If you are working at home, make sure you have the version of BlueJ we've customized at OCC. (Ask me for help if necessary.) Create a new project in your home directory with a name of your choice (maybe hw09). A new BlueJ project is initially empty, containing no classes.

   b. Create a new class using the "New Class" button. Give your class the name "CollectBeepers" and choose the "Karel robot task class" template.

   c. Double-click on the newly created class to open an edit window. Consider resizing it to make it longer, so you can see more of the code. Read the comments carefully (they are in green). Fill in the "@author" and "@version" information in the first comment block near the top of the file.

d. Near the bottom of the file, you will see two comments in red. The first is on a line like this:

```
World.startFromURL( /*# place URL for world description here */ );
```

Replace everything between the parentheses with this URL in double quotes: "http://csjava.occ.cccd.edu/~gilberts/worlds/hw09.kwld". Your program will be able to read the description of the initial world state right off the course web site.

e. Compile your program. Fix any syntax errors. After successful compilation, create an instance of your class on the object bench (refer to the BlueJ tutorial, Chapter 3, available from BlueJ's help menu if you don't remember how to do this). Right-click on your instance and invoke the `task()` method to see what happens (you may need to move windows around and bring Karel's world window to the front in order to see it). You have just executed your first method call on a live object.

2. Create a robot within your class

a. The remaining comment asks you to "add your statements here". Delete it and replace it with this line to create a new robot:

```
karel = new OCRobot();
```

b. Compile your class, create a new instance of it on the object bench, and invoke the task() method again. What is different?

3. Navigate to the first beeper

a. Choose which beeper you want to pick up first. Add method calls to orient and move `karel` until the robot is over your chosen beeper. You can recompile your class and re-run the `task()` at any time to check your progress. You can also adjust the speed control that is visible when Karel's world window is open if you want the robot to move faster (initially, it runs at the slowest speed).

b. Direct `karel` to pick up the beeper.

4. Pick up the remaining beepers

a. Devise a path through the remaining beepers. Add method calls to direct `karel` to navigate to each beeper and pick it up in turn. Feel free to re-compile and re-run your robot task at any time to check your progress.

b. Once you have added enough method calls to cause `karel` to pick up all the beepers, add another to turn the robot off.

Part II: Testing

In Part II of this lab assignment, you will create your own tests to demonstrate that your robot does what is required.

Procedure

1. Create a class to hold your tests

a. First, make sure that BlueJ's testing features are enabled. click Tools->Preferences... from BlueJ's main menu, then select the Miscellaneous tab. Make sure that "Show unit testing tools" is checked (it is at the bottom, under "Unit test settings"). Click "OK".

b. Right-click your existing class and select "Create Test Class". This will create a new class to hold your tests. It will have the same name as the original, with "Test" added on the end. you can add as many tests to this one class as you like.

2. Create a test fixture

a. A "test fixture" is a fancy name for a collection of objects that you want to check against some criteria. Normally, that means the objects have already been created, and that you've already sent them the methods needed to get them into the state you want to check. We'll create three objects, all on the object bench. First, create an instance of your main task class on the object bench. Then execute its `task()` method so that it creates your robot and then directs it to complete its work.

b. Now, we want to get a reference to your robot on the object bench. Right-click your task object and invoke the new `robot()` method. In the dialog that follows, BlueJ will show a reference to your `karel` object. Select it in the list, then click the "Get" button to add your robot to the object bench.

c. Finally, we want to get a reference to the robot's world on the object bench. Right-click your robot object. You'll see all of its basic methods available. Choose the "inherited from `TestableRobot`" submenu, then invoke the `getWorldAsObject()` method (it may be under a "more..." submenu). As before, "Get" the resulting object reference to add the robot's world to the object bench.

d. Right-click the test class you created. Select "Object Bench to Test Fixture." This will record all of the objects you have on the object bench inside the test class. Now, each test that you add to this test class will be run with the current configuration of objects as its starting point. In other words, each test will be executed with three objects available, in the state after your robot task has been executed.

3. Create a test case

a. A "test case" is one particular test you want to check. Normally, a test case includes a statement of what actions to perform, together with a statement of how to check whether those actions had the desired effect. Here, you've already done the first part in setting up the test fixture. Now you need to decide what to check. First, let's check that `karel` has some beepers.

b. Right-click your test class and select "Create Test Method...". Give your method a meaningful name indicating what it checks. You will notice that your object bench reverts to the stored "test fixture" state for this test class and the red "recording" light on the left of BlueJ's main window comes on.

c. Any actions you take now are being "recorded" as part of your test case. Right-click on your robot, and under "inherited from `TestableRobot`", invoke the `assertBeepersInBeeperBag()` method.

d. Finish your test case by clicking "End" on the left of BlueJ's main window. Your test class should be recompiled automatically. Open the test class in an edit window and look through its source. Can you read it? While you have the test class open, go ahead and fill in the comments at the top of the class, just like you did with your regular robot task class.

4. Run your test

a. You can run your tests by right-clicking on your test class and selecting "Test All" (run all tests in this class), or using the "Run Tests" button on the left of BlueJ's main menu (run all tests in all test classes). You can also run individual tests one at a time by right-clicking on a test class. Run your test now.

b. Look at the "Test Results" dialog that opens. It lists your tests one by one in the top half, together with a "progress bar" that moves left to right as each test is executed. The bar will turn red if any test case fails. Clicking on the test case in the top half of the window will produce a description of what/how it failed in the bottom half.

5. Add more tests

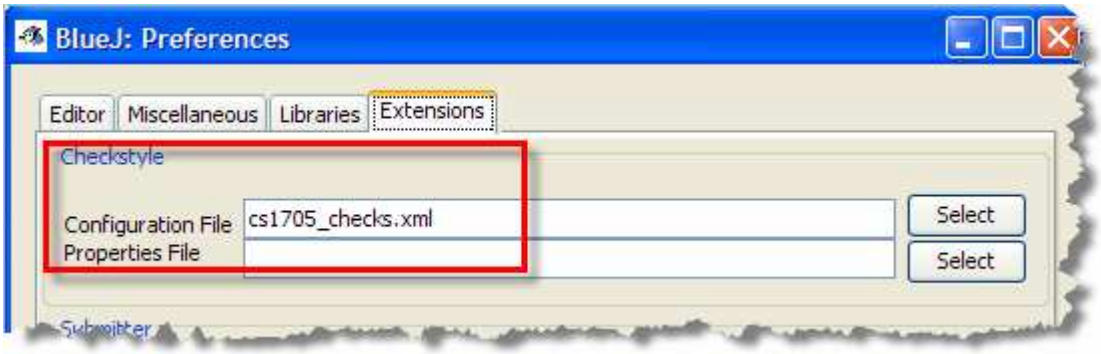
a. Think about what else you'd like to check to ensure that `karel` did the job required. Brainstorm with other students. Come up with at least three other conditions that will help ensure Karel did his job. Note that the robot object provides many methods useful for "checking" conditions in a test case. These methods are often called **assertions**, and are named using a pattern like `assert<Condition>()`, for checking whether some condition is true. You can invoke these under the "inherited from TestableRobot" submenu available from your robot object. There is a similar "inherited from TestableWorld" submenu available from the world object, and it provides numerous assertions about the state of the world (rather than the state of the robot itself).

b. Practice recording new test cases for the checks you have come up with. Run each test after recording it. You can also directly edit the test class source code to modify or add to an existing test case, or even to add entirely new test cases. Remember that all of the test cases are evaluated with the same collection of objects in the same state--the "test fixture". To write tests for a different collection of objects as well, usually you need to create a new test class. Also, when running lots of tests, the "speed control" that is visible when Karel's world window is shown is very useful--try turning the speed to the fastest possible speed.

c. Create a test case that fails. Run it. What happens? How can you tell what went wrong in that test?

6. Check your Style

Make sure that you have first followed the lecture and configured the Checkstyle plug-in by adding cs1705_checks.xml on the Checkstyle configuration page:



Then, on BlueJ's main menu, select Tools->Checkstyle... from the main menu. Check the style for the CollectBeepers class and make sure you fix any issues. You don't have to correct the issues the Checkstyle points out in your test class (mostly missing Javadoc statements) since that file was automatically written for you.

7. Submit your assignment

Create a PDF document named **LastFirst_hw09.pdf.** Your homework should have your name on the top of the page along with these items, in this order:

a. A screenshot of your completed robot task after all of the beepers have been collected and the robot shut off

b. A screenshot of the JUnit test window showing that you've passed your tests as expected.

c. A screenshot of the Checkstyle window showing no problems.

d. Copy and paste the source code for both your CollectBeepers and your test program into your document.

See the class Web page for instructions on submitting your assignment.